

```
parent.html | Input.html | active | x
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title><Create ASCII Art</title>
6 </head>
7 <body>
8 {% extends "parent.html" %}
9 {% block content %}
10
11 {% if ascii|'valid' %}
12 <h2>Converted Image </h2>
13 <div id="ascii-art">
14 {%- for line in ascii|'img' %}
15 {{line}}<br>
16 {%- endfor %}
17 </div>
18 {% else %}
19 <h2 class="urlMsg">Invalid URL </h2>
20 <h3 class="urlMsg">{{ ascii|'img' }}</h3>
21 {% endif %}
22 <a href="{{ url_for('home') }}">Home</a>
23
24 {% endblock %}
25 </body>
26 </html>
```

Code
Collapse

HTML Templates

Create the Parent Template

Before we start with Python code, let's create the parent template and then the two children templates. This is a rather simple webpage. It uses the `style.css` file for styling, has an `<h1>` tag with the title, and has a single content block. The content block is inside a `<div>` with the class name `'container'`.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title><Create ASCII Art</title>
</head>
<body>
<div class="title"><Create ASCII Art</div>
</div>
<div class="container">
{%- block content %}
{%- endblock %}
</div>
</body>
</html>
```

Input Template

Open the `input.html` page. This will serve as the entry point for the user. They will enter a URL and click a button to convert it to ASCII art. Start by extending `parent.html` and set up a content block. In the block, create an `<h2>` tag with instructions to enter an image URL.

Open `input.html`.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title><Create ASCII Art</title>
</head>
<body>
{%- extends "parent.html" %}
{%- block content %}
<h2> Enter the URL of an image </h2>
{%- endblock %}
</body>
</html>
```

Add a form with a text field and a submission button. Under the action, have it use the `/ascii` route (which we will define later). Give the form the name `url` so we can reference it. Set `url` as the value. That way this text will appear in the text box. However, we want this text to disappear when the user clicks on the text box. Set `onfocus` to `"this.value=''"`. This will replace the text in the text box with an empty string when clicked. Give each `<input>` tag a unique ID for styling. In addition, add a `<details>` tag (which creates text hidden behind a dropdown menu) explaining how users can right-click on an image to copy its URL. Give this text a `op` tag and the `"finding"` ID for styling.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title><Create ASCII Art</title>
</head>
<body>
{%- extends "parent.html" %}
{%- block content %}
<h2> Enter the URL of an image </h2>
<form action="{{ url_for('ascii') }}" method="POST">
<input type="text" id="urlbox" name="url" value="Image URL" onfocus="this.value=''">
<input type="submit" id="urlbutton" value="Create">
</form>
<details>
<summary><strong>How do I get an image URL?</strong></summary>
<p id="finding">Right-click on an image. You should see a window pop up. The</p>
</details>
{%- endblock %}
</body>
</html>
```

Art Template

The `art.html` template is the page users see after clicking the button generate ASCII art. Use the link below to open the art template.

Open `art.html`.

Like the above template, extend `parent.html` and create a content block in the `<body>`. Add a link to the content block that takes the user back "home", that is the `input.html` page.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title><Create ASCII Art</title>
</head>
<body>
{%- extends "parent.html" %}
{%- block content %}
<a href="{{ url_for('home') }}">Home</a>
{%- endblock %}
</body>
</html>
```

What we show here depends on the result of the `create_ascii` function we made at the beginning of the lab. Either it returns a string with a message about entering a valid URL or it returns a list of strings. We can create conditionals in Flask with the `{% if %}` statement. This code assumes Python passes a dictionary to the code block. The dictionary has two key-value pairs. The first key is `'valid'`, which contains a boolean value. If the `create_ascii` function returns ASCII art, the value is `True`. Add a for loop to the HTML document that iterates over the list stored in `'img'` key of the dictionary. Add each element from the list to the webpage followed by a `<br>` tag so each row of characters is on its own line. Place the ASCII art in a `<div>` with the ID `ascii-art` for styling purposes.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title><Create ASCII Art</title>
</head>
<body>
{%- extends "parent.html" %}
{%- block content %}
{%- if ascii|'valid' %}
<h2>Converted Image </h2>
<div id="ascii-art">
{%- for line in ascii|'img' %}
{{line}}<br>
{%- endfor %}
</div>
{%- else %}
<h2 class="urlMsg">Invalid URL </h2>
<h3 class="urlMsg">{{ ascii|'img' }}</h3>
{%- endif %}
<a href="{{ url_for('home') }}">Home</a>
{%- endblock %}
</body>
</html>
```

If the `valid` key is `False`, then add `{% else %}` before `{% endif %}`. Create a title as an `<h2>` tag and add an `<h3>` tag with the string message from the `create_ascii` function. The key for this information is called `img`. Both of these elements should have the class `urlMsg` for styling purposes.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title><Create ASCII Art</title>
</head>
<body>
{%- extends "parent.html" %}
{%- block content %}
{%- if ascii|'valid' %}
<h2>Converted Image </h2>
<div id="ascii-art">
{%- for line in ascii|'img' %}
{{line}}<br>
{%- endfor %}
</div>
{%- else %}
<h2 class="urlMsg">Invalid URL </h2>
<h3 class="urlMsg">{{ ascii|'img' }}</h3>
{%- endif %}
<a href="{{ url_for('home') }}">Home</a>
{%- endblock %}
</body>
</html>
```

Did you notice?

Since we were not interacting with Python, we did not start the Conda virtual environment. You must activate the virtual environment when working with Python as it has all of the required packages installed. It is not necessary to edit HTML pages. However, you must activate the virtual environment to run our Flask app and see the HTML pages rendered in the browser.

Lab Question

Assume that `age` is an integer variable used in Flask. Select the correct way to determine if `age` is greater than 21.

- ☒ `{% if age > 21 %}`
`>older than 21</p>`
`{% endif %}`
- ☐ `{% if age > 21 %}`
`>older than 21</p>`
`{% end %}`
- ☐ `{% if age > 21 %}`
`>older than 21</p>`
`{% endif %}`
- ☐ `{ if age > 21 }`
`>older than 21</p>`
`{ endif }`

The correct answer is:

- ```
{% if age > 21 %}
>older than 21</p>
{% endif %}
```
- Start a conditional with an `if` statement that has `{% if %}` at the beginning and `{% %}` at the end
  - After the conditional is through, end it with `{% endif %}`

Submit

Next